

Soft Error Rate Evaluation and Silent Data Corruption Detection for Reliable TFHE Computations on GPUs

Masakazu Yoshida[✉], Kotaro Matsuoka, and Masanori Hashimoto[✉], *Senior Member, IEEE*

Abstract—Fully homomorphic encryption (FHE), which enables computations to be performed on encrypted data, is a promising approach to privacy-preserving cloud computing. However, reliability, particularly with respect to cosmic ray-induced errors, becomes a concern because FHE requires substantial computational time and resources, and error detection in encrypted data is inherently challenging. In this paper, we evaluate the terrestrial soft error rate of a GPU-based implementation of Fully Homomorphic Encryption over the Torus (TFHE), a variant of FHE that supports logical operations, via accelerated proton irradiation experiments. Our measurements show that the error rate in a terrestrial environment is 7.80 FIT (Failures In Time) for the decrypted output of a 32-bit adder. Assuming that the overall error rate scales proportionally with circuit size, we estimate a silent data corruption (SDC) probability of 1.6% for the multiplication of 500-by-500 matrices. To address these concerns, we propose two methods to detect SDCs. One approach, which leverages logic circuit-level redundancy, can detect nearly all SDCs, at the cost of reducing throughput to around 50%.

Index Terms—Soft error, TFHE, homomorphic encryption, reliability, silent data corruption (SDC), GPU.

I. INTRODUCTION

CLOUD computing, which provides computing resources as a service, continues to gain widespread popularity as the information society continues to evolve [1], [2]. In this context, exchanging sensitive data, such as personal information, between users and service providers must be encrypted to ensure confidentiality. Moreover, it is crucial that even cloud system administrators not have access to this data. However, in conventional systems where only communication is encrypted, the ciphertext is ultimately decrypted on cloud servers for processing, which can unintentionally expose confidential information. In addition to system administrators exploiting their privileges or attackers exploiting operating system vulnerabilities, the physical nature of devices introduces the risk of side-channel attacks, which can leak sensitive data [3]. Consequently, there is a growing need for cloud services that leverage fully homomorphic encryption (FHE) [4], enabling arbitrary computations without ever decrypting the data.

Received 23 January 2025; revised 26 June 2026; accepted 11 July 2026. Date of publication 17 July 2026; date of current version 1 September 2026. This work was supported in part by the JST CREST under Grant JPMJCR19K5 and in part by the Grant-in-Aid for Scientific Research (S) from Japan Society for the Promotion of Science (JSPS) under Grant 24H00073. (Corresponding author: Masanori Hashimoto.)

The authors are with the Department of Informatics, Kyoto University, Kyoto 606-8501, Japan (e-mail: hashimoto@i.kyoto-u.ac.jp).

Digital Object Identifier 10.1109/TDSC.2026.3714766

For the practical application of FHE-based cloud services, both high computational speed and high reliability are essential. FHE over the Torus (TFHE) [5], a type of FHE discussed in this paper, can perform arbitrary logic operations; however, its first CPU implementation [6] requires 13 ms to evaluate a single logic gate [5], leading to very long processing times for logic circuits of practical scale. This presents a major obstacle to the widespread adoption of FHE-based systems. Consequently, various hardware platforms, including CPU, GPU, Application Specific Integrated Circuit (ASIC), and Field Programmable Gate Array (FPGA), have been explored to accelerate these systems [7], [8], [9], [10], [11].

To ensure the practical reliability of TFHE-based cloud services, two fundamentally different error sources must be considered:

- *Cryptographic noise*: Homomorphic operations inherently accumulate noise in ciphertexts, which may cause decryption errors. In TFHE, however, this algorithmic error is mitigated by bootstrapping after every gate evaluation.
- *Hardware-induced errors*: The hardware executing the computation may experience transient hardware faults, which can cause system crashes or silent data corruptions (SDCs).

While cryptographic noise is controlled algorithmically, hardware-induced errors remain a serious threat.

One major cause of unreliability in computer systems is soft errors from cosmic ray-induced particles striking semiconductor devices. Cosmic rays originating from space interact with oxygen and nitrogen atoms in the atmosphere, producing neutrons that continuously reach the Earth's surface. Although the probability of a soft error occurring is very low, such errors can become a serious issue in large-scale data centers that operate thousands of servers continuously. In cryptographic computations, even a single-bit error in ciphertext can prevent decryption and is difficult to detect. Because such undetected errors can corrupt data silently without triggering hardware or software alarms, they are called Silent Data Corruption (SDC) [12]. Evaluating both the impact and frequency of soft errors is therefore vital for cryptographic reliability. However, the soft error rate (SER) for FHE remains unexamined. FHE often relies on large-scale cloud servers, which include substantial hardware, such as memory and cores, susceptible to soft errors that degrade reliability [13], [14]. Hence, validating FHE-based applications on real systems is crucial before deploying FHE for oblivious inference [15] or database services [16].

GPUs were originally developed for graphics rendering but have evolved into essential components for applications requiring high computational parallelism, particularly in the context of general purpose graphics processing units (GPGPU), thanks to their availability and well-established programming environments. Recently, they have been adopted in safety-critical systems, such as autonomous vehicles, where even minor computational errors can lead to significant consequences [17]. As a result, research on the reliability of GPU-based applications has expanded considerably, particularly in fields like convolutional neural networks (CNNs) for image recognition [18]. As GPUs continue to be integrated into reliability-critical environments, ensuring their accuracy and robustness against external factors has become a top priority.

In this paper, we evaluate the SER of TFHE applications running on a GPU on the ground through accelerated irradiation experiments of high-energy protons, where a GPU is chosen as the platform for TFHE in this work due to its high availability and achievable throughput thanks to parallel homomorphic gate evaluations. For SER measurements, it is common practice to perform accelerated tests, in which the number of incident particles is increased to obtain sufficient statistics in a shorter time, and the measured results are then scaled down to the terrestrial environment. To the best of our knowledge, this is the first work to address soft error concerns in FHE and to perform SER evaluation for TFHE as one of the representative FHE schemes. We also measured the SER of on-chip GPU memory and a TFHE-based logic circuit, verifying whether the experimental results can be reproduced via fault injection simulation. The overall fluence for the above accelerated experiments was $8.7 \times 10^{10} \text{ p/cm}^2$, where the unit denotes the number of protons per square centimeter. Furthermore, because the intermediate values in FHE computations remain encrypted, conventional plaintext-based checks, e.g., range checks, cannot be applied on the cloud server, making SDC particularly difficult to detect. To address this challenge, we propose two methods for detecting SDCs and quantitatively discuss their characteristics to improve the reliability of TFHE applications. The major contribution of this work is summarized as follows:

- *First terrestrial SER evaluation for TFHE:*
This work presents a first assessment of the terrestrial SER in TFHE applications running on a commercial GPU, using proton accelerated irradiation experiments. The results suggest that a circuit performing the multiplication of 10-by-10 matrices under TFHE experiences SDCs at a rate of 0.41%, which can be prohibitively high.
- *Proposal of SDC detection methods:*
Recognizing that intermediate FHE values remain encrypted and drawing on unique features of TFHE, this work proposes two different approaches for detecting SDC. One method utilizes logic circuit-level redundancy, while the other exploits the phase information in the encrypted data. The approach relying on circuit-level redundancy can detect nearly all SDCs, albeit at the cost of reducing throughput to about 50%.

It should be noted that cosmic-ray-induced soft errors occur in both client and server machines under terrestrial

conditions. However, this study focuses on servers, as TFHE computations—especially bootstrapping and gate evaluations—are highly intensive and typically executed on cloud or data-center servers with continuous large-scale parallel operations.

The remainder of this paper is as follows. Section II reviews related work and clarifies the context of this work. Section III provides essential background. Section IV presents our terrestrial SER evaluation for TFHE. Section V discusses proposed SDC detection methods and analyzes performance. Finally, Section VI summarizes the main conclusions.

II. RELATED WORK

A. Fault Tolerance of Cryptography

A considerable body of research has investigated fault tolerance in cryptographic systems, with a focus on the encryption [12], [19], [20], [21], [22], [23], [24], [25], transmission [12], [22], [23], and decryption [21], [22], [23], [24], [25] stages. However, each of these phases remains susceptible to external factors such as radiation, fault injection attacks, and general system noise.

Following the standardization of AES by NIST [26], numerous studies have examined its vulnerability to soft errors. For example, Banu et al. proposed an error propagation model for AES [12], while subsequent efforts explored methods to detect and correct these errors both algorithmically and at the FPGA implementation level [19], [20], [21]. In parallel, attacks on RSA [27] and corresponding countermeasures have been thoroughly investigated, given RSA's foundational role in digital signatures and authentication [22], [23], [24], [28], [29]. With the expansion of the Internet of Things (IoT), lightweight cryptographic algorithms have been widely researched, prompting efforts to assess and mitigate soft errors in terrestrial applications [25]. Research has also been extended to cross-cutting methods for error detection and correction that are not limited to specific cryptographic frameworks [23].

Despite these advances, hardware-oriented fault tolerance in FHE remains insufficiently explored. In contrast to conventional cryptographic schemes, FHE includes an additional homomorphic computation phase on encrypted data. Therefore, fault tolerance in FHE must be analyzed not only for encryption and decryption, but also for homomorphic evaluation itself.

Most prior work on homomorphic encryption (HE)/FHE has focused on correctness from algorithmic and cryptographic perspectives, including noise growth analysis, parameter selection, and bootstrapping [30], [31]. More recently, compiler- and IR-level techniques have also been proposed to automate parameter selection and to preserve or verify the correctness of homomorphic programs [32], [33]. The purpose of these studies is to ensure decryptability and semantic correctness under the HE/FHE computation model. These studies are complementary to fault-tolerance research, but they do not address transient hardware faults during execution.

Our work addresses a different aspect of reliability. We focus on transient hardware faults arising during practical TFHE execution on GPU platforms. This problem is largely orthogonal to algorithmic noise accumulation: even if the cryptographic noise

is properly managed, hardware-induced faults may still lead to crashes or silent data corruption.

Furthermore, prior work on HE/FHE accelerators and run-times has primarily focused on performance optimization, memory efficiency, and scheduling [34], [35]. Recent studies have also begun to address resilience and fault-related issues in FHE systems [36], [37]. In particular, ReliaFHE proposes a framework to protect both storage and computation units in future dedicated FHE accelerators such as ASICs, whereas our work focuses on practical countermeasures for existing GPU platforms. GlitchFHE, by contrast, studies adversarial fault injection attacks on FHE accelerators. Our work differs from these studies in that it experimentally characterizes naturally occurring hardware-induced soft errors during practical TFHE execution on GPUs and proposes TFHE-specific detection methods.

B. SER Evaluation and Hardening Techniques for GPUs

Commercial devices must be rigorously assessed for reliability before they are adopted in applications that demand high dependability. Accordingly, researchers have been exploring soft error evaluation methods and mitigation strategies across a wide range of devices and applications. Existing studies target various domains, including space applications [38], [39], [40], terrestrial safety-critical systems [18], [41], and high-performance computing (HPC) [14], [42].

GPUs have many functional units and run numerous threads concurrently, making them prone to soft errors. A single error in a shared resource (e.g., the scheduler or shared memory) can propagate widely, causing disruption [18], [41]. GPU reliability has been studied using RTL [43], microarchitectural [44], and software-based [45], [46], [47] fault injection, beam experiments [18], and hybrid approaches [44], [48]. While beam experiments are highly accurate, they are expensive, making fault injection appealing. However, because memory available for injection is limited in commercial devices, it is essential to verify how closely these results reflect real-world system behavior.

Some studies focus on enhancing the reliability of GPU applications. Duplication with comparison strategies are simple and effective, making them a practical starting point for exploring fault tolerance in applications where reliability is still an emerging area of research [14]. Although the duplication with comparison introduces processing overhead, it provides a straightforward and algorithm-independent approach to ensure reliability and can facilitate initial progress in the field. Besides, for well-established algorithms such as matrix multiplication and fast Fourier transform (FFT), many studies have already proposed and evaluated algorithm-based fault tolerance (ABFT), demonstrating its efficiency in reducing overhead [14], [18], [49], [50]. However, in this paper, we focus on applications where such advancements have not yet been extensively explored.

TFHE, the focused application of this paper, demands substantial computational resources. Large servers contain vast amounts of hardware, including memory, which increases the probability of single- or multi-bit upsets during the computation

process and reduces overall reliability [13], [14]. Consequently, it is essential to evaluate these reliability challenges and implement appropriate countermeasures as needed.

III. BACKGROUND

A. Soft Error

Soft errors are temporary malfunctions caused by cosmic rays that occur in integrated circuits [51]. The main phenomenon is bit flips in memory elements, which cause program malfunctions or calculation errors. Single event upset (SEU) is a temporary error in a circuit that results from a state change of one of more cells, combinatorial cells and storage cells, due to an energetic particle strike [51]. An SEU can lead to the occurrence of single bit upset (SBU), multiple-bit upset (MBU), or other types of errors.

In the context of system behavior analysis, errors are commonly categorized as SDC or Detected Unrecoverable Error (DUE) [52]. An SDC is a computational error that produces an unexpected output without raising any alerts or symptoms. Detecting SDC is challenging when results differ across executions or when the correct output is unknown in advance. In contrast, a DUE is an error that leads to a crash or other form of abnormal termination. In scenarios where continuous operation is critical, DUE becomes a major concern.

When discussing the SER of a particular device or system in a particular environment, failure in time (FIT), the number of failures per 10^9 hours, is often used as a unit [51]. The cross section (σ) is defined as the number of observed events of interest (N_e) divided by the total particle fluence (Φ), namely, $\sigma = N_e/\Phi$, where N_e is the number of events of interest observed during irradiation, and the fluence Φ is the number of incident particles per unit area (typically expressed in cm^{-2}). The unit of σ is therefore cm^2 per device (or per bit), and it quantifies the probability that a single particle strike will induce an event of interest. Hence, a larger cross section corresponds to a higher probability of event occurrence. By multiplying the cross section of the device or system and the particle flux in the environment, we obtain the SER. Meanwhile, when discussing the propagation of errors, the architectural vulnerability factor (AVF) is used. The AVF of a program is the probability that a fault in a processor results in a visible error in outputs [53].

It has been reported that the frequency of such occurrences increases as semiconductor devices shrink and memory capacity grows [54]. Research is ongoing to explore physical configurations that enhance server reliability and balance performance and reliability [55], [56].

B. TFHE

TFHE is a type of FHE that allows an arbitrary number of logical operations to be performed on ciphertext without decryption [5]. In the remainder of this paper, we refer to these logic operations as HomGates and to circuits composed of HomGates, which correspond to TFHE applications, as HomCircuits. When specifying a particular gate type, we use terms like HomNAND, HomXOR, and so forth.

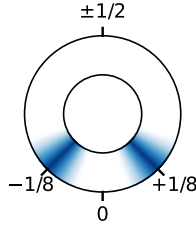


Fig. 1. Distribution of TLWE phases in Torus.

TABLE I
TRUTH TABLE OF HOMNAND ON THE TORUS, WHERE $x, y \in \mathbb{T}$. NOTE THAT ACTUAL TFHE COMPUTATIONS ARE PERFORMED UNDER ENCRYPTION.

x	y	$1/8 \cdot \text{sgn}(1/8 - x - y)$
+1/8	+1/8	-1/8
+1/8	-1/8	+1/8
-1/8	+1/8	+1/8
-1/8	-1/8	+1/8

We here describe the TFHE theory related to this paper with the following notations.

- $\mathbb{B}$: The set $\{1, 0\}$ without any structure.
- $\mathbb{T}$: The set of real Torus $\mathbb{R}/\mathbb{Z}$, the set of real numbers modulo 1. In this paper, we define the interval of $\mathbb{T}$ to be $[-0.5, 0.5)$.
- $\mathbb{Z}^+$: The set of positive integers.
- $\mathbb{R}^+$: The set of positive real numbers.

TLWE is the learning with error (LWE) ciphertext [57] using Torus coefficients. The ciphertext in TLWE format, which is the input/output of HomGate, is an $(n+1)$ -dimensional Torus array denoted by (a, b) . In the encryption process, $b = a \cdot s + \mu \cdot (2m - 1) + e$ is calculated. Note that $s \in \mathbb{B}^n$ is the secret key, $m \in \mathbb{B}$ is the plaintext, and $e \in \mathbb{T}$ is the noise. e and a are random variables that follow modular gaussian distributions with standard deviation α , and uniform distributions, respectively. The parameters $n \in \mathbb{Z}^+$ and $\alpha \in \mathbb{R}^+$ determine the strength of the cipher. In the decryption process, $(1 + \text{sgn}(b - a \cdot s))/2$ is computed to obtain the plaintext m . Note that $\text{sgn}(x)$ is a function that is equal to 1 when $x > 0$ and -1 otherwise. $b - a \cdot s$ is called a phase. The parameters used in this paper are $n = 635$, $\alpha = 2^{-15}$, $\mu = 1/8$. Therefore, the values of TLWE, including noise, are distributed around $\pm 1/8$ as shown in Fig. 1.

Logic gates over TFHE can be performed by addition and subtraction of phases. For example, Table I shows the truth table of HomNAND(x, y), which is equivalent to $1/8 \cdot \text{sgn}(1/8 - x - y)$ in plaintext, where $+1/8$ and $-1/8$ correspond to logical 1 and 0, respectively. Other types of 2-input logic gates are implemented in a similar way, changing the sign of each term to achieve various gates. HomMUX and HomNOT are implemented in slightly different ways for efficiency. HomMUX can be implemented with three 2-input HomGates straightforwardly, because MUX is expressed as $(s \wedge x) \vee (\neg s \wedge y)$, where x is chosen if s is true, and y otherwise. Because at least one of $(s \wedge x)$ and $(\neg s \wedge y)$ is false, $\vee$ is replaced with addition. Therefore, the computation of HomMUX can be

simplified as

$$\begin{aligned} \text{HomMUX}(s, x, y) &= \text{HomAND}(s, x) \\ &\quad + \text{HomAND}(\neg s, y) + (0, 1/8), \quad (1) \end{aligned}$$

which needs only two 2-input HomGates. HomNOT just inverts the sign of coefficients of input TLWE.

In TFHE, bootstrapping is performed for every single HomGate evaluation. Each bootstrapping operation removes the noise that has accumulated during the HomGate evaluation, thereby maintaining the decryptability of ciphertexts. In this bootstrapping, an arbitrary multi-bit function, typically 4-bit, can be embedded, allowing flexible evaluation of nonlinear functions such as compound gates [58] or lookup tables [59]. However, in this paper, we consider only the standard sign function (sgn), which is the most fundamental and commonly used function in TFHE for HomGate evaluation.

Evaluating HomCircuits is well-suited to GPU-based implementations with high parallel processing capabilities, as HomGates without dependencies can be evaluated in parallel. In this paper, we assume a parallel HomGate evaluation scheme on GPUs using cuFHE.

C. GPU Architecture

In recent years, GPUs have evolved into high-performance devices for HPC and are used in some of the world's leading supercomputers [60]. Additionally, they are developed as System-on-a-Chip (SoC) solutions for power-constrained applications such as mobile devices. This section provides an overview of the NVIDIA GPU programming model and its physical architecture [61].

First, we introduce the programming model. When using NVIDIA GPUs, implementations are typically done in the CUDA programming language [62]. GPUs operate according to a hierarchical structure, outlined as follows:

- *Grid*: A collection of Blocks
- *Block*: A collection of Threads
- *Thread*: The smallest unit of execution for a program

When implementing a CUDA program, users specify the number of Blocks and Threads to execute in parallel, enabling the desired level of parallelism. For instance, in matrix-related computations, individual operations on matrix elements are often independent. In such cases, executing operations in parallel for all elements at Block-level or Thread-level, can significantly enhance computational performance.

Next, the physical architecture of NVIDIA GPU is explained. GPUs comprise numerous streaming multiprocessors (SMs), global memory, L2 cache, memory controllers, and other components. Each SM consists of several CUDA cores, L1 cache, register file, etc. The Blocks specified in the program correspond to SMs, while Threads map to CUDA cores. To achieve the specified degree of parallelism, tasks are allocated to SMs and CUDA cores on the GPU. The GPU architecture includes both on-chip and off-chip memory. On-chip memory, implemented as Static Random Access Memory (SRAM), includes the register file, shared memory, L1 cache, and L2 cache, while off-chip

memory, implemented as Dynamic Random Access Memory (DRAM), consists of global memory.

IV. SOFT ERROR RATE EVALUATION

In this section, we evaluate the soft error rate of TFHE applications via irradiation and fault injection experiments, and discuss the impact of soft errors. First, Section IV-A introduces the error classification used in our experiments. Next, Sections IV-B and IV-C detail our experimental setup and present the measured error cross sections. In Section IV-D, we assess the reproducibility of these results using fault injection. Finally, Section IV-E assesses the SER of TFHE applications in terrestrial environments.

A. Error Classification for TFHE

During TFHE operations, faults caused by SEUs can occur during encryption, computation, or decryption. This paper focuses on faults arising during computation, as computation typically occurs on cloud servers, whereas encryption and decryption are often performed on client-side machines. Besides, there are two primary types of errors that may occur: calculation errors and abnormal program terminations.

The first type, calculation errors, commonly result from bit flips in registers or caches that store the computation results. This leads to incorrect outputs but normal program termination. Such errors are referred to as SDC. Because cryptographic operations are highly sensitive, even a single-bit error in a HomGate computation can produce a drastically different outcome, making correct plaintext recovery impossible. Moreover, detecting these errors mid-computation is challenging, as conventional techniques (e.g., value range checks) cannot be directly applied to encrypted data.

The second error type is known as a detected unrecoverable error (DUE), which manifests as abnormal program termination or a system hang. For instance, a bit flip in the memory address calculation may trigger illegal memory access, or a bit flip in the instruction cache can cause the execution of undefined instructions. Although DUEs may require a system reboot, the errors are at least detected, allowing the system to re-compute the task. Furthermore, because TFHE applications inherently involve substantial computational overhead, recomputation can be a viable option. Consequently, SDC poses a more significant concern, and its detection is crucial for improving the reliability of TFHE applications.

In this paper, we focus on discussing the SDCs in which the output values after decryption are flipped. Table II shows the SDC classification. We evaluate the errors at two major hierarchies: a logic gate over TFHE and a circuit over TFHE. For the former, we classify the gates affected by the soft error as HomGate-level SDC and HomGate-level Potential (Pot.) SDC based on whether its decrypted output is flipped or not. For the latter, we classify the circuits including gates affected by the soft error as HomCircuit-level SDC and HomCircuit-level Pot. SDC based on whether at least one of its decrypted primary outputs is flipped.

TABLE II
HOMGATE- AND HOMCIRCUIT-LEVEL SDC CLASSIFICATION

	SDC	Potential (Pot.) SDC
HomGate-level	The plaintext output bit is flipped.	The encrypted output exhibits faults, but the plaintext output remains error-free.
HomCircuit-level	One or more bits in the plaintext primary output are flipped.	One or more HomGate-level SDCs occur within the HomCircuit, but these have no impact on the plaintext primary output.

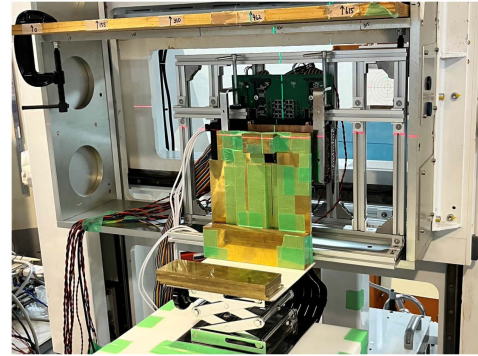


Fig. 2. DUT on the proton beamline. We irradiated only the GPU chip using a metal collimator.

B. Proton Irradiation Experiments

To evaluate the terrestrial soft error rate, accelerated proton irradiation experiments were conducted at the Wakasa Wan Energy Research Center in Japan [63]. This section provides details of these experiments. It should be noted that we have confirmed error-free DUT operation under non-irradiated (no-beam) conditions, and that the observed errors originated from protons.

Although most particles reaching the ground and causing soft errors are neutrons, we used protons in this study. Previous research shows that neutron irradiation of DRAMs can induce not only transient soft errors but also stuck bits, a form of persistent error [64], [65], [66]. Consequently, to isolate and evaluate the impact of soft errors only on the GPU chip, we chose proton beams, which can be easily shielded with a metal plate. According to JESD89B [51], protons with energies of 50 MeV or higher can serve as substitutes for neutrons of equivalent energy. We used 96 MeV proton beams; hence, we expect our measured error rates to be comparable to those observed under neutron irradiation.

The device under test (DUT) was an NVIDIA RTX A4000, which has 48 SMs [67]. Fig. 2 shows the DUT setup. The DUT is positioned behind brass and copper collimators so that only the GPU chip is exposed to the collimated proton beam. The GPU card and motherboard with an Intel i5-12400F processor are connected via a 60 cm PCIe cable, with the motherboard located well below the beam (outside the photo). Other components, such as the power supply, are placed alongside the motherboard, away from the beam.

TABLE III
ERROR CATEGORIES FOR ON-CHIP MEMORY, FOLLOWING THE JESD89B
CLASSIFICATION [51]

Category	Description in this paper
Single bit upset (SBU)	A single bit flip is observed in a readout.
Multiple-bit upset (MBU)	Two or more bit flips are observed in a readout.
Reset soft error	Require re-run but no need to reboot
Power cycle soft error	Require reboot

Before conducting the main experiments, we carried out a preliminary test to determine a suitable proton flux. A flux that is too high prevents the host computer from booting, whereas a flux that is too low yields insufficient error samples. Therefore, we carefully selected the highest flux of $2.6 \times 10^6 \text{ p/cm}^2/\text{s}$ that could still emulate realistic conditions as closely as possible. More specifically, the flux was adjusted so that approximately one out of several thousand HomCircuit evaluations would stochastically result in an SDC or DUE in this experiment.

We measured the SEU cross sections of the GPU on-chip memory, which will be used for fault injection in Section IV-D, as well as the SDC and DUE cross sections for individual HomGates and a HomCircuit consisting of HomGates. The cuFHE library [68], [69] and Iyokan [70] were used for these experiments, where Iyokan was customized for logging purposes. The evaluation of HomGates with cuFHE is conducted using multiple SMs, where each HomGate evaluation uses one SM, and the evaluation of a HomCircuit, which is parallel evaluation of HomGates, is scheduled and controlled by Iyokan. For other TFHE libraries, similar computations are performed at the HomGate level, and therefore the discussion in this paper is expected to remain applicable. The following subsections describe the setup and procedure for each measurement.

1) *SEU Cross Sections of On-Chip Memories in GPU*: The target components in this experiment are the register files and shared memory. An RTX A4000 features 65536 register files and a combined 128 KiB of L1 data cache/shared memory per SM. For our measurement, we focus on 60 KiB of register file storage and 48 KiB of shared memory per SM.

During irradiation, we repeatedly run two CUDA programs; one testing the register files and the other testing the shared memory. Each program writes data once and then continuously reads it until an error is detected. Table III presents the error categories used in this experiment, as defined by JESD89B [51]. The program logs SBU, MBU, and reset soft errors. If the program hangs, we categorize the error as power cycle soft error and reboot both the DUT and the host computer.

2) *SDC Cross Sections of HomGates*: The HomGate types evaluated in this experiment are HomNAND, HomXOR, and HomMUX. Ideally, all HomGate types would be tested; however, due to practical constraints, such as irradiation costs, we narrowed down the selection. In addition, while 2-input HomGates share similar implementations, their computations differ from those of 1- and 3-input gates. Therefore, we chose one or two gates from each category, for the following reasons:

- 2-input NAND is selected because it is a basic logic gate and the set {NAND} is functionally complete.

- 2-input XOR is selected because it is commonly used in arithmetic circuits and is included in the 32-bit adder (our primary HomCircuit of interest).
- 2-input, 1-select MUX is selected as one of 3-input gates, as it is frequently used in data paths.
- NOT is excluded because its implementation is extremely simple, merely negating all coefficients, and involves an instruction count that is five orders of magnitude smaller than that of other 2-input HomGates. As a result, HomNOT is significantly less susceptible to soft errors.

The experimental procedure is as follows:

- 1) Make a secret key and a bootstrapping key at the host and transfer the bootstrapping key to the GPU.
- 2) Generate HomGate inputs in plaintext randomly at the host.
- 3) Encrypt the inputs with the secret key at the host.
- 4) Evaluate a HomGate 48000 or 24000 times by repeating the following sub-steps.
 - 1) Transfer a set of the encrypted inputs from the host to the GPU.
 - 2) Evaluate the HomGate on the GPU.
 - 3) Transfer the encrypted outputs from the GPU to the host.
- 5) Decrypt all the outputs with the secret key and classify the decrypted outputs as HomGate-level SDC or no error at the host.

We recorded both the decrypted output of the HomGates and any CUDA error messages. To maximize GPU utilization, the number of HomGates evaluated per execution, i.e., the number of Step 4 repetitions, should be large. However, if it is too high, the likelihood of a DUE increases, causing more executions to be discarded and reducing the number of valid HomGate evaluations for counting SDC. To balance this trade-off, we ran HomNAND and HomXOR 48000 times each, and HomMUX 24000 times per execution. With this setup, a single execution takes approximately 30 seconds to complete, and 80% of the executions are valid, which will be shown later.

3) *SDC Cross Sections of HomCircuit*: The HomCircuit used in this experiment, representing a TFHE application, is a 32-bit adder composed of 188 HomGates (102 HomNANDs, 63 HomXORs, and 23 HomNOTs). We selected this circuit for its AVF, indicating that SDCs at the HomGate level are likely to propagate and affect the HomCircuit output.

The experimental procedure is as follows:

- 1) Make a secret key and a bootstrapping key at the host and transfer the bootstrapping key to the GPU.
- 2) Encrypt inputs with the secret key at the host.
- 3) Transfer the encrypted inputs and HomCircuit netlist from the host to the GPU.
- 4) Evaluate the HomCircuit on the GPU using Iyokan.
- 5) Transfer the encrypted output of all HomGates from the GPU back to the host.
- 6) Decrypt all outputs with the secret key and classify whether the circuit output contains HomCircuit-level SDC, and whether each HomGate output contains HomGate-level SDC.

Iyokan issues one HomGate evaluation as a CUDA block, that is, from a hardware perspective, one HomGate is processed by

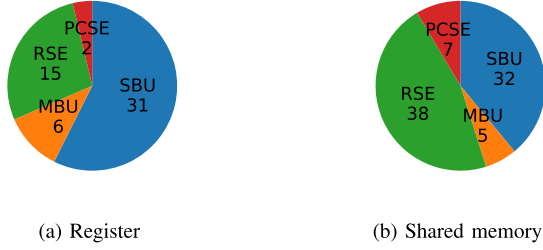


Fig. 3. The numbers and proportions of error categories observed in the on-chip memory experiment. RSE and PCSE are the categories of reset soft error and power cycle soft error, respectively.

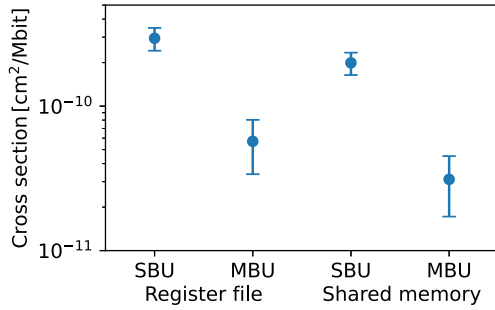


Fig. 4. SBU and MBU cross sections of register file and shared memory. Error bars represent the standard deviations of the corresponding Poisson distributions.

one SM, taking into account input-output dependencies in the HomCircuit to maximize parallelism [8].

C. Experimental Results

1) *SEU Cross Sections of On-Chip Memories in GPU:* Fig. 3(a) and (b) show the proportion of each error category, as defined in JESD89B [51] and listed in Table III. Reset soft errors and power cycle soft errors were also observed in our experiments and were not negligible in some cases. However, since this paper mainly focuses on the analysis of SDC resulting from SBUs and MBUs, these error categories are excluded from the following discussion.

Fig. 4 shows the cross sections of SBU and MBU in the register file and shared memory. While the SEU cross section (encompassing both SBU and MBU) appears higher in the register file than in the shared memory, the difference is inconclusive due to the error bars. The obtained cross sections will be referenced in the fault injection experiment in Section IV-D.

2) *SDC Cross Sections of HomGates:* We categorized each execution into three classes: no error (neither SDC nor DUE observed), DUE (DUE observed), and SDC (SDC observed but no DUE). Fig. 5 shows the proportions of these three classes. Note that if a DUE occurs during an execution, no SDC can be observed in that same execution. However, by adjusting the number of HomGate evaluations per execution as mentioned earlier, we controlled the DUE ratio so that it would not become excessively large. The subsequent discussion will focus on the observed SDCs.

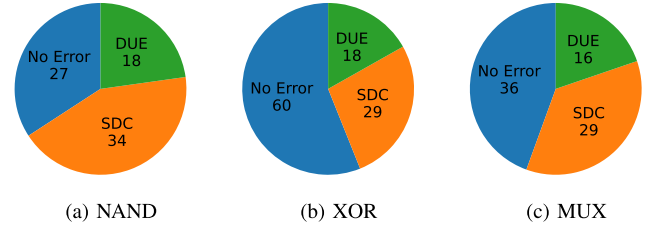


Fig. 5. The numbers and proportions of error categories observed in the HomGate experiment.

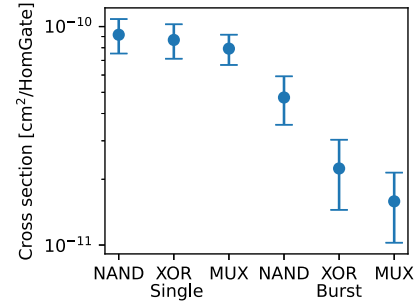


Fig. 6. HomGate-level SDC cross sections. Error bars represent the standard deviations of the corresponding Poisson distributions.

Fig. 6 illustrates the HomGate-level SDC cross sections for single HomGate evaluations. Here, an SDC is defined as any incorrect plaintext value obtained after decryption, as specified in Table II earlier. A burst SDC error is one in which a large number of SDCs occur consecutively in time. More precisely, any sequence of errors with indexes separated by fewer than 96 steps, twice the number of SMs, is considered a burst, whereas any SDC outside this criterion is deemed a single SDC error. The index here refers to the evaluation sequence of Step 4. Since the DUT has 48 SMs, and HomGates with an index of $48k + i$ ($k \geq 0$) are assigned to the i -th SM, this criterion allows HomGate-level SDCs and Pot. SDCs occurring simultaneously on different SMs to be treated as a single phenomenon as well. While a burst consists of multiple SDCs, it is treated as one event for calculation of the cross section, which differs from counting the number of HomGates that produce incorrect plaintext outputs.

Fig. 6 shows that the cross sections for single SDC ranged from $7.93 \times 10^{11} \text{ cm}^2/\text{HomGate}$ to $9.18 \times 10^{11} \text{ cm}^2/\text{HomGate}$. The error bars for all three HomGate types overlap each other, indicating that they have a similar vulnerability. The reason for the similar cross sections for the 2-input HomGate (NAND, XOR) is thought to be that the TFHE calculations are almost the same, even for different logic operations. The calculation process of three-input HomMUX includes two 2-input HomAND calculations as explained in Section III-B, and inherently HomMUX is expected to have the cross section similar to those of 2-input HomGates. Meanwhile, the cross sections of burst SDC ranged from $1.59 \times 10^{11} \text{ cm}^2/\text{HomGate}$ to $4.74 \times 10^{11} \text{ cm}^2/\text{HomGate}$. HomNAND and HomXOR have

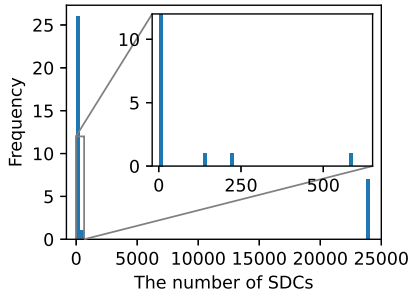


Fig. 7. Distribution of the number of HomGate-level SDCs in each execution (48000 runs for HomNAND).

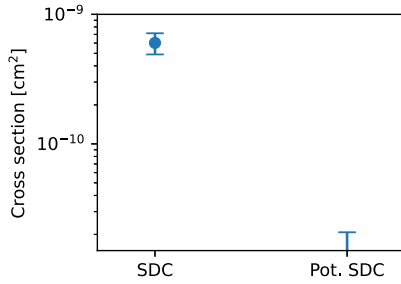


Fig. 8. HomCircuit-level SDC cross sections. Error bars represent the standard deviations of the corresponding Poisson distributions. The error bar of Pot. SDC corresponds to one event since no events were observed.

overlapping error bars and similar cross sections, while HomMUX had a slightly smaller error rate than the other two HomGates. In the case of HomMUX computation described by (1), if one of the HomAND gates is inverted, the HomMUX output is also inverted. However, if both HomAND gates are inverted, the output remains normal. Therefore, we consider that a smaller cross section than NAND and XOR was observed.

Fig. 7 shows the number of HomGate-level SDCs observed in each HomNAND execution. The distributions exhibit two peaks, near 0 % and 50 % of the total HomNAND evaluations per execution. The lower peak ranges from 100 to 600 SDCs, while the higher peak ranges from 23800 to 24200 SDCs. A 50% rate indicates that nearly all encrypted outputs are affected, resulting in 50% plaintext errors (SDCs) and 50% potential SDCs. Although the exact causes of burst SDCs remain unclear, possible explanations include smaller-scale impact within a single block or larger-scale effects stemming from issues such as a malfunctioning block scheduler or a corrupted bootstrapping key in the GPU on-chip memory. From a detection standpoint (discussed in Section V), burst SDCs are easier to detect than single SDCs. Therefore, in this study, we place greater emphasis on single SDCs.

3) *SDC Cross Sections of HomCircuit*: Fig. 8 shows the cross sections of the 32-bit adder HomCircuit. Recall that HomCircuit-level SDC refers to errors resulting in bit flips in the plaintext primary output, while HomCircuit-level Pot. SDC indicates one or more HomGate-level SDCs within the circuit that do not propagate to the primary output. In our experiment,

the HomCircuit-level SDC cross section was $6.02 \times 10^{10} \text{ cm}^2$, and the HomCircuit-level Pot. SDC cross section was 0 cm^2 . The HomCircuit used in this study is an adder circuit, which inherently has the property that any change in a logic gate output is likely to affect the circuit output. For example, in a ripple-carry adder consisting of full adders, if an intermediate logic value changes, the final addition result will also change. This explains the absence of Pot. SDC events in this experiment.

In contrast, general logic circuits often exhibit logic masking, where errors occurring in some gates do not propagate to the final output. Therefore, in general cases, the HomCircuit-level potential SDC cross section would not be zero.

4) *Key Observations*: We summarize the key findings in the experiments for HomGate and HomCircuit.

- The HomGate-level SDC cross sections are similar across NAND, XOR, and MUX, indicating comparable vulnerability regardless of logic type.
- HomMUX shows a slightly smaller SDC rate due to logical redundancy when both internal AND gates are inverted.
- Burst SDCs appear as a distinct mode, affecting up to 50% of outputs, likely due to large-scale GPU faults or corrupted bootstrapping keys.
- The 32-bit adder HomCircuit exhibited no potential SDCs in our experiment because any gate-level upset is likely to propagate to the primary output, unlike general logic circuits where logic masking can often occur.

D. Estimation of SDC Cross Section With Fault Injection

In this section, we investigate the reproducibility of SDC cross sections between irradiation and fault injection experiments. The section is divided into two parts: the first focuses on HomGate, and the second on HomCircuit.

1) *HomGate*: HomGate-level SDC cross section (σ_G) can be estimated as follows:

$$\sigma_G = \sum_{m \in \text{Memory}} A_m \sigma_m C_m, \quad (2)$$

where A_m represents the AVF of HomGate for memory m , σ_m represents the cross section of memory m , and C_m represents the capacity used in the evaluation of HomGate. Here, AVF is defined as the probability of an SDC occurring at the HomGate output when a fault is injected. In this paper, we consider the register file, shared memory, and L1 cache as m . The global memory is not included since the DRAM was not irradiated in the experiment.

First, we obtain the memory capacity used in HomGate evaluations. The average number of live registers can be checked using NVIDIA Nsight Compute [71], and was found to be 36.8 per thread. Because each HomGate uses 768 threads in parallel, the average number of registers used is $36.8 \times 768 = 2.83 \times 10^4$ per HomGate. The amount of shared memory used is 56 KiB per HomGate. We assume that the amount of L1 cache used by HomGate is 72 KiB since a SM has a unified architecture for shared memory and L1 cache and its capacity is 128 KiB.

Next, we evaluate the AVF of HomGates using NVBitFI [45], a tool for injecting transient faults into GPU kernels. NVBitFI

TABLE IV
THE AVF OF HOMGATE

Gate\Memory	Register	L1 cache	Shared
NAND	0.42	0.33	0.37
XOR	0.46	0.09	0.51
MUX	0.42	0.15	0.36

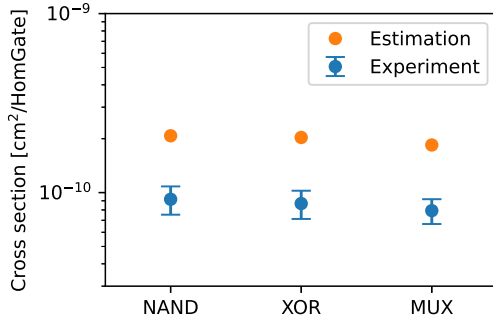


Fig. 9. Comparison of experimental results and estimation.

can inject the following types of faults into the destination register values:

- *Single bit-flip*: one bit-flip in one register in one thread
 - *Double bit-flip*: bit-flips in two adjacent bits in one register in one thread
 - *Random value*: random value in one register in one thread
 - *Zero value*: zero out the value of one register in one thread
- Additionally, the following instruction groups can be targeted:
- GP: Instructions that write to general purpose registers. GP includes FP32, FP64, LD, and other instructions
 - FP32: Single precision floating point instructions
 - FP64: Double precision floating point instructions
 - LD: Load instructions

For our evaluation, we used the single bit-flip model and targeted the GP instruction group to evaluate A_m . Because the original NVBitFI targets an older GPU architecture, we adapted it for the Ampere architecture [72]. While the injected instructions are randomly selected, we classify them into the following three categories for A_m characterization of the register file, L1 cache and shared memory, respectively:

- Instructions that take two operands from registers and write the result to a destination register.
- Instructions that load from global memory via L1.
- Instructions that load from shared memory.

We injected 2000 faults and summarized the AVF of target memories in Table IV.

Using the results in Fig. 4, we estimate the SDC cross sections shown in Fig. 9, assuming the SEU cross section of the L1 cache is the same as that of shared memory. This assumption holds because shared memory and the L1 cache are mapped to the same physical memory in our DUT. Although these SDC cross sections are overestimated by a factor of about 2–2.5, they still serve as a reasonable approximation. Besides, in actual environments, certain bit flips occur in memory regions that will not be read before the next write. However, NVBitFI cannot

TABLE V
COMPARISON OF EXPERIMENTAL RESULTS AND ESTIMATION

Experiment	Estimation	Error
6.02×10^{-10}	2.71×10^{-10}	-55.0%

easily reproduce such scenarios, which likely contributes to the overestimation.

2) *Homcircuit*: HomCircuit-level SDC cross section (σ_C) is estimated as follows:

$$\sigma_C = (A_{C,\text{single}} \cdot \sigma_{G,\text{single}} + A_{C,\text{burst}} \cdot \sigma_{G,\text{burst}}) N, \quad (3)$$

where $A_{C,\text{single}}$ and $A_{C,\text{burst}}$ are the average AVF for single and burst SDCs, respectively, $\sigma_{G,\text{single}}$ and $\sigma_{G,\text{burst}}$ are the average cross section of single and burst SDCs, respectively, and N is the average number of HomGates running in parallel. In this context, AVF represents the probability that a HomGate-level SDC will propagate to the primary output of the HomCircuit without being logically masked.

N is determined by the circuit structure and the number of SMs in the GPU. The HomCircuit requires 62 time slots for evaluation and includes 165 two-input HomGates, giving $N = 165/62 = 2.66$. The remaining 23 HomGates are Hom-NOT gates, which compute much faster and are, therefore, negligible in this calculation. To obtain $A_{C,\text{single}}$, we performed a logic-level fault injection with the following steps:

- 1) *Target selection*: Randomly select one gate from the target circuit.
- 2) *Injection*: Replace the selected gate with its complement (e.g., NAND $\rightarrow$ AND, XOR $\rightarrow$ XNOR) and evaluate the resulting circuit. We used the same input values (0x96323ff5 and 0x783e17e9) as those in the irradiation experiment.

From these experiments, we determined $A_{C,\text{single}} = 0.75$ for all gates based on fault injection results. Because a burst SDC has a significant impact on primary outputs, we assume $A_{C,\text{burst}} = 1$. Using the results shown in Fig. 6, we calculate $\sigma_C = 2.7 \times 10^{-10} \text{cm}^2$ as listed in Table V. Although this estimate appears reasonably acceptable, the observed error of -55.0% is still somewhat large, possibly because the limited beam time in the irradiation experiment restricted the number of observable events. Another possible reason is that the assumption that σ_C is proportional to N in (3) may introduce some inaccuracy, as certain hardware components are shared among SMs.

E. Assessing Soft Error Rate in a Terrestrial Environment

To discuss the SDC occurrence frequency in TFHE applications, we assess the soft error rate in a terrestrial environment. Assuming a high-energy terrestrial neutron flux of $3.6 \times 10^{-3} \text{n/cm}^2/\text{s}$ [51], and treating the error cross sections for protons and neutrons as equivalent, we estimate that the HomGate-level SDC rates per HomGate at ground level range from 1.03 FIT to 1.19 FIT for single SDCs and from 0.205 FIT to 0.614 FIT for burst SDCs. Meanwhile, the HomCircuit-level SDC rate for the 32-bit adder is estimated to be 7.80 FIT. Note

that neutron and proton cross sections above 50 MeV are quite similar and relatively constant. However, at energies below 50 MeV, the neutron cross section is lower than the proton cross section at 96 MeV. Accordingly, the SDC rates reported here can be considered upper bounds.

Here, we project 7.80 FIT to a large-scale AI GPU cluster and estimate how frequently SDCs may occur. Referring to reports that 16,384 H100 GPUs were utilized for LLaMA 3.1 training [73], we estimate the SDC rate for a GPU cluster consisting of 16,384 H100 GPUs. Given that the H100 and RTX A4000 have 14,592 and 6,144 CUDA cores, respectively, we scale the FIT value proportionally to the number of CUDA cores. The resulting SDC rate is estimated to be 0.000304 errors per hour ($= 7.80 \times 14592 / 6144 \times 16,384 \times 10^{-9}$), corresponding to approximately 0.219 SDC errors per month.

To explore more general and practical TFHE applications, we discuss the reliability of matrix multiplication under the following conditions and assumptions:

- *Target:* Multiplication of two n -by- n matrices (32-bit)
- *Required arithmetic operations:* n^3 multiplications and $n^3 - n^2$ additions
- *Arithmetic hardware on TFHE:* 32-bit array multipliers and 64-bit adders, where each 32-bit array multiplier consists of thirty-one 32-bit adders (AND gates for partial product generation are ignored), and each 64-bit adder consists of two 32-bit adders. We experimentally verified through fault injection that the 32-bit and 64-bit adders exhibit a 0% logical masking effect. Therefore, we assume that the SER of the 64-bit adder is twice that of the 32-bit adder.
- *Measured results used:* 7.80 FIT and 1.74 s execution time for the 32-bit adder.

Under these conditions, we estimated the probability including SDCs—more precisely, the expected number of errors per matrix multiplication—as $7.80 \times 10^{-9} \times 1.74 / 3600 \times (31n^3 + 2(n^3 - n^2))$ for $n = 100$ and $n = 500$. It should be noted that the actual FIT rate of matrix multiplication depends on factors such as the parallel computing capability of the GPU card and the degree of parallelism in logic gate evaluations. In fact, the FIT rates of 1.03 to 1.19 FIT for a single HomGate and 7.8 FIT for a HomCircuit are not proportional to the circuit size. The resulting estimated probabilities are 0.012% for $n = 100$ and 1.6% for $n = 500$. These values are relatively high for commercial applications, indicating that in large-scale TFHE-based systems, the impact of soft errors is expected to be noticeable.

The estimated probabilities above are determined by the FIT rate, the execution time of the 32-bit adder, and the total computation amount. To achieve reliable large-scale computation, both the FIT rate and the execution time must be reduced. These goals can be realized through custom accelerator designs dedicated to TFHE, as such hardware is memory-efficient and significantly reduces computation time (e.g., [74], [75]). Consequently, these custom accelerator designs contribute not only to lower latency but also to enhanced reliability.

Although the estimated per-HomCircuit SER may appear relatively low, it can become practically significant in large-scale TFHE workloads because many HomCircuit evaluations are executed over a long computation time. Thus, the impact of

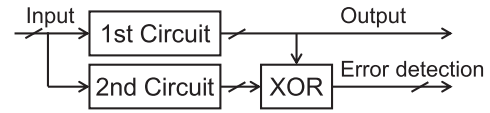


Fig. 10. Redundant circuit including error detection signal.

soft errors accumulates with workload size, as shown in the matrix multiplication estimates above. This does not imply that full redundancy is always necessary; rather, the appropriate countermeasure depends on the application scale and reliability requirement. This motivates the SDC detection methods and the reliability–performance trade-off analysis discussed in Section V.

V. SDC DETECTION METHODS

Based on the practical implications discussed in the previous section, we propose and evaluate SDC detection methods to enhance the reliability of TFHE applications.

A. Proposed Methods

This section proposes two methods to detect SDCs in HomCircuits.

1) *SDC Detection by Logic-Level Redundancy:* The first method, **Method #1**, exploits logic-level redundancy by duplicating the HomCircuit and then comparing the primary outputs using HomXORs to detect errors (Fig. 10). The procedure is as follows:

- 1) The servers evaluate the duplicated circuit over TFHE.
- 2) The servers send both the encrypted primary output and the encrypted error detection signal to the user.
- 3) The user decrypts the error detection signal using their secret key and decides whether to accept or reject the result, i.e., the received primary output.

Because the HomXOR outputs remain encrypted, whether they are 0 or 1 is unknown until decryption. Method #1 effectively performs the HomCircuit evaluation twice and adds extra gates (HomXOR), reducing computation throughput to about 50% (or less) compared with a normal evaluation.

2) *SDC Detection Using Phase Distribution:* The second method, **Method #2**, exploits the fact that TLWE noise follows a modular Gaussian distribution, illustrated in Fig. 1 for the torus without errors. Accordingly, if the raw phase of a HomGate output after the phase addition and subtraction but before bootstrapping deviates from the distribution shown in Fig. 1, we infer that soft errors have occurred. This raw phase information is obtained during decryption. However, note that any raw phase misalignment will be erased by the consequent bootstrapping process in the HomCircuit evaluation.

- 1) Servers evaluate the normal HomCircuit.
- 2) The servers send all HomGate outputs to the user.
- 3) The user decrypts these outputs with their secret key, extracts and examines the raw phases, and decides whether to accept or reject the result. They check each HomGate output raw phase individually. If one or more raw phases

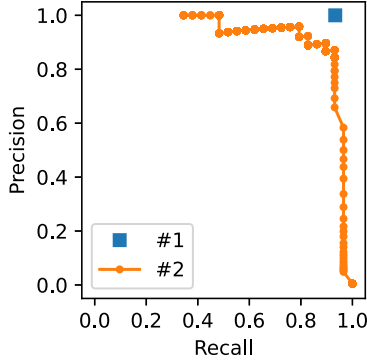


Fig. 11. Precision-recall curves of Methods #1 and #2.

lie outside the range defined by k times the standard deviation, the HomCircuit evaluation is assumed to have been affected by soft errors. Here, k is a user-defined parameter.

The parameter of k can be selected according to user requirements, taking into account the precision–recall tradeoff. Since decryption requires the user’s secret key, only the user can determine whether a failure has occurred.

B. Evaluation

We applied the proposed methods to the experimental results and examined the performance of each method. For this purpose, we also evaluated the duplicated HomCircuit in the irradiation experiment described earlier. For Method #1 evaluation, the results of the duplicated HomCircuit evaluation are used below.

Fig. 11 shows the precision-recall curves of the proposed methods. The parameter k is swept from 1×10^{-20} to 1 for Method #2. In the precision-recall curve, proximity to the top-right corner indicates higher classification performance. Method #1 achieved the highest performance in the precision-recall tradeoff, but it requires twice the computational resources compared to cases without any countermeasures. On the other hand, Method #2 does not increase HomGate evaluation, but requires additional processing, specifically decrypting all HomGate outputs to obtain their phases, compared to normal execution.

C. Missed SDCs

Since Method #1 demonstrated strong performance in the precision-recall curve, we now focus on this method and examine the probability at which SDCs are missed in each HomCircuit evaluation. The probability P_{missed} that an SDC is not detected by Method #1 is expressed as follows:

$$P_{\text{missed}} = \sum_{i,j \geq 0} P_{i,j} F_{i,j}, \quad (4)$$

where $P_{i,j}$ is the probability that i and j HomGate-level SDCs occur in the first and second HomCircuits, respectively. Note that the HomXORs for comparison are included in the second HomCircuit. $F_{i,j}$ is the probability that both HomCircuits

exhibit HomCircuit-level SDC and the error detection signal remains zero when i HomGates in the first HomCircuit and j HomGates in the second HomCircuit experience SDC. In other words, $F_{i,j}$ represents the probability of failing to detect an error when there are i SDCs in the first HomCircuit and j SDCs in the second HomCircuit. Accordingly, the summation of $P_{i,j} F_{i,j}$ in (4) represents the overall error missing probability per HomCircuit evaluation with Method #1. In the following paragraphs, we calculate (4), considering only single HomGate SDCs because burst SDCs are easier to detect than single SDCs.

First, $P_{i,j}$ can be expressed as $P_{i,j} = P_i^{(1)} P_j^{(2)}$ because single SDCs occur randomly. Here, $P_i^{(1)}$ is the probability that there are i SDCs in the first HomCircuit and is given by:

$$P_i^{(1)} = \sum_{\sum_g x_g = i} \left[\prod_{g \in G} p_g^{x_g} \binom{N_g^{(1)}}{x_g} \right], \quad (5)$$

where p_g denotes the HomGate-level SDC occurrence probability during type- g HomGate evaluation, x_g denotes the number of SDC of type- g HomGates, and $N_g^{(1)}$ denotes the number of type- g HomGates in the first HomCircuit. We introduce these type-wise computations to account for type-dependent SDC rates, and G represents the set of HomGate types with distinct SDC rates. The number of all HomGates in the first circuit is $N^{(1)} = \sum_g N_g^{(1)}$. Similarly, $P_j^{(2)}$, the probability that there are j SDCs in the second HomCircuit, can be computed in the same way as (5).

Let us apply the P_{missed} computation to the 32-bit adder used in the irradiation experiment. The target HomCircuit consists of HomNAND, HomXOR, and HomNOT gates. Based on Fig. 6, we assume that HomNAND and HomXOR share the same error rate, and HomNOT does not incur any SDC for the reason described in Section IV-B2. Consequently, $P_i^{(1)}$ can be simplified as follows:

$$P_i^{(1)} = p_A^i \binom{N_A^{(1)}}{i}, \quad (6)$$

where group A consists of HomNAND and HomXOR gates. The HomGate-level SDC probability p_A for group A is expressed as $p_A = \sigma_{G,A} \times \text{Flux} \times \text{Time}$, where $\sigma_{G,A}$ is the cross section of group-A HomGates.

Second, $F_{i,j}$ is calculated via a logic-level circuit simulation. We conducted 10000 simulation runs according to the following steps:

- 1) *Target selection*: We randomly choose i and j gates of the target circuit.
- 2) *Injection*: We replaced each selected gate with its complement and simulated the modified circuit.

The resulting values of $F_{i,j}$ are shown in Table VI. As the number of SDCs occurring in a single execution increases, their occurrence probability decreases significantly, and their impact on the result becomes less significant. Therefore, we consider the condition $i + j \leq 5$.

By setting $N_A^{(1)} = 165$, $N_A^{(2)} = 165 + 32$, and $p_A = 9.0 \times 10^{-15}$, we obtain a P_{missed} of 3.8×10^{-26} in the terrestrial

TABLE VI

THE PROBABILITY OF FAILING TO DETECT AN ERROR WHEN THERE ARE i AND j SDCs IN THE 1ST AND 2ND HOMCIRCUIT FOR THE DUPLICATED 32-BIT ADDER. THE VALUE OF $F_{i,j}$ ($i + j \leq 5$) IS DISPLAYED IN THE i -TH ROW AND j -TH COLUMN.

1st \ 2nd	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0.0145	0.0067	0.0016	0.0013	
2	0	0.0085	0.0060	0.0027		
3	0	0.0049	0.0024			
4	0	0.0023				
5	0					

environment. Consequently, Method #1 reduces the SDC probability from 3.9×10^{-12} (separately calculated for the case without SDC detection methods) to 3.8×10^{-26} , representing a substantial improvement in reliability.

D. Comparison With Secure MPC

It is worth discussing an alternative error-handling paradigm often associated with secure multi-party computation (MPC), another prominent approach for privacy-preserving systems. MPC and garbled-circuit-based approaches have been explored for practical secure computation and secure neural-network inference, as well as for hardware/software co-designed acceleration of garbled circuits [76], [77], [78]. In some MPC-based or garbled-circuit-based systems, the reconstructed or decoded result may be checked against expected validity conditions, such as output range, format, or consistency constraints. As a result, erroneous computations can often be detected after reconstruction or decryption without requiring continuous redundancy during execution, enabling a relatively low-cost detect-and-retry strategy in some settings.

In contrast, TFHE fundamentally differs in this respect. Since TFHE ciphertexts ultimately encode only binary values (0 or 1), there is little explicit redundancy in the decrypted value itself. An erroneous computation may therefore still decrypt to an apparently valid 0 or 1, making simple output-level validity checks ineffective. This lack of explicit redundancy motivates the dedicated error-detection mechanisms proposed in this work. That said, Method #2 can also be interpreted as exploiting a form of internal redundancy in TFHE representations. Although the logical value is still only 0 or 1, ciphertexts corresponding to the same logical value may carry different noise phases. Method #2 leverages this latent phase information to detect abnormal computations. In this sense, it is conceptually related to detect-and-check strategies used in secure computation, but is realized within the representational constraints unique to TFHE.

This comparison also clarifies the role of the trade-off analysis below: because TFHE lacks simple output-level validity checks, improving reliability requires explicit mechanisms whose detection capability, computational overhead, and applicability depend on the target workload and reliability requirements.

E. Trade-Off Analysis and Discussion

The appropriate SDC detection method should be selected according to the criticality of the target application. In safety-critical applications, even a single undetected error may lead to

TABLE VII

RELATIVE COSTS OF TWO PROPOSED METHODS, NORMALIZED BY THE COSTS OF THE ORIGINAL HOMCIRCUIT EVALUATION. HERE, N_{gc} REPRESENTS THE NUMBER OF HOMGATES IN THE ORIGINAL HOMCIRCUIT, AND N_{oc} IS THE NUMBER OF PRIMARY OUTPUTS IN THE ORIGINAL HOMCIRCUIT.

	Original	Method #1	Method #2
Computation at servers	1	$2 + N_{oc}/N_{gc}$	1
Computation at users	1	2	N_{gc}/N_{oc}
Communication from servers to users	1	2	N_{gc}/N_{oc}

serious consequences. On the other hand, users of TFHE-based computation should recognize that, even under normal terrestrial conditions, soft errors may occur with a certain probability, as discussed in Section IV-E. Therefore, the acceptable level of redundancy or performance degradation should be determined according to the required reliability of the application.

A cost comparison of the proposed methods is summarized in Table VII. Here, the costs for server-side computation, user-side computation, and communication are each normalized to those of the original HomCircuit evaluation. Method #1 requires more than a twofold increase in GPU computation on the servers. In addition, users must decrypt both the primary outputs and the error-detection signals. In contrast, Method #2 does not increase the computation cost on the servers, but it requires users to decrypt all HomGate outputs for SDC detection, whereas only the primary outputs need to be decrypted in the original scenario. Since all communicated data must be decrypted by the users, the communication cost is equivalent to the user-side computation cost.

To complement the normalized comparison in Table VII, Table VIII shows the measured wall-clock execution time and communication volume for 32-bit and 64-bit adders. In these experiments, the user-side decryption time was less than 0.001 s, and the difference among the methods was not measurable. On the server side, Method #1 increased the execution time, although the increase was smaller than a factor of two even when the circuit size doubled probably due to the parallel computation. By contrast, Method #2 showed nearly the same server runtime as the original HomCircuit evaluation. However, Method #2 substantially increased the communication volume because all intermediate gate outputs had to be transmitted to the user. These results indicate that Method #1 mainly introduces additional computational overhead on the server side, while Method #2 mainly shifts the overhead to communication bandwidth.

Cost alone, however, is not sufficient to determine the preferable method; detection capability must also be considered. As shown in Fig. 11, Method #1 offers better precision-recall characteristics. Moreover, the discussion in Section V-C shows that Method #1 achieves an extremely low probability of missed errors in a terrestrial environment. Therefore, when additional server-side computation is acceptable, Method #1 is the preferable choice for applications requiring strong reliability.

By contrast, Method #2 may be attractive when preserving server throughput is more important than maximizing detection accuracy. Since SDC events are relatively infrequent, occasional rejection and recomputation may be acceptable in some applications, even when a certain number of false positives are

TABLE VIII
ABSOLUTE COSTS OF TWO PROPOSED METHODS

DUT		Original			Method #1			Method #2		
Adder [bit]	#Hom- Gates	Time [s]		Comm. [kiB]	Time [s]		Comm. [kiB]	Time [s]		Comm. [kiB]
		Server	User		Server	User		Server	User	
32	165	3.138	<0.001	80.5	6.403	<0.001	161	3.145	<0.001	415.1
64	340	4.715	<0.001	161	9.485	<0.001	322	4.692	<0.001	855.3

introduced. In such cases, the parameter k should be selected according to the required trade-off between error detection and overhead. Although the user-side decryption time was negligible for the evaluated adders, this overhead may become non-negligible for larger HomCircuits. In that case, selectively decrypting only a subset of gates, rather than all gates, may be a practical extension of Method #2.

It should be noted that for Method #1, partial or selective redundancy is feasible (e.g., [79]). Applying the redundant Method #1 only to critical or error-sensitive gates might achieve a trade-off between reliability and performance. To achieve higher detection accuracy with minimal overhead, it will be essential to investigate algorithm-based fault tolerance (ABFT), which provides algorithm-specific optimizations and improved efficiency. ABFT techniques have been extensively studied in GPU applications (e.g., [14], [18], [49], [50]), and future advancements in this area could be incorporated into reliable HomGate evaluation. Another promising direction is computation using redundant representations (e.g., [80], [81]). We also expect that more efficient computation methods for TFHE—at the algorithmic, hardware, and software levels—will continue to be developed. As discussed in Section IV-E, efficient and fast TFHE computation contributes to reliable TFHE applications. Further exploration of the reliability–performance trade-off across various workloads and future hardware platforms will be included in our future work.

VI. CONCLUSION

This paper presented an evaluation of the SER for GPU memories and TFHE-based applications running on a GPU, using accelerated proton irradiation experiments. We found that the 32-bit adder HomCircuit exhibited a HomCircuit-level SDC rate of 7.80 FIT under terrestrial conditions. Furthermore, our results suggest that the multiplication of 500-by-500 matrices is likely to experience SDCs with a probability of 1.6%. As circuits become larger and more parallelized, the impact of soft errors is expected to escalate, making reliability a critical concern for future TFHE implementations. To address this issue, we proposed and evaluated two distinct methods for detecting SDCs. Each method offers a different balance of detection performance and computation overhead, allowing system designers to select the option best suited to their specific requirements. Adopting the most appropriate detection strategy will be essential for ensuring reliable and robust TFHE computations at scale.

ACKNOWLEDGMENT

The authors thank R. Banno, a developer of the Iyokan, for his support for the experiments. The authors also thank the

Kyoto University Entrepreneurs Platform (KUEP) for its support for the experiments. The authors used OpenAI's ChatGPT to assist in language editing and improving the readability of the manuscript. All technical content and conclusions were verified by the authors.

REFERENCES

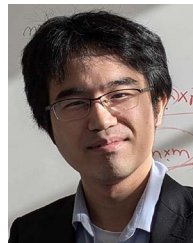
- [1] P. Mell and T. Grance, "The NIST definition of cloud computing," NIST Special Publication 800-145, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Sep. 2011.
- [2] L. Heilig and S. Vob, "A scientometric analysis of cloud computing literature," *IEEE Trans. Cloud Comput.*, vol. 2, no. 3, pp. 266–278, Third Quarter 2014.
- [3] S. Van Schaik et al., "SoK: SGX.fail: How stuff gets eXposed," in *Proc. 2024 IEEE Symp. Secur. Privacy*, May 2024, pp. 4143–4162.
- [4] C. Gentry, "A fully homomorphic encryption scheme," Ph.D. dissertation, Stanford Univ., Stanford, CA, USA, 2009.
- [5] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds," in *Advances in Cryptology—ASIACRYPT 2016*, J. H. Cheon and T. Takagi, Eds. Berlin, Germany: Springer, 2016, pp. 3–33.
- [6] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast fully homomorphic encryption library," Aug. 2016. [Online]. Available: <https://tfhe.github.io/tfhe/>
- [7] K. Matsuoka, "TFHEpp: Pure C implementation of TFHE cryptosystem," 2020. [Online]. Available: <https://github.com/virtualsecureplatform/TFHEpp>
- [8] K. Matsuoka, R. Banno, N. Matsumoto, T. Sato, and S. Bian, "Virtual secure platform: A five-stage pipeline processor over TFHE," in *Proc. 30th USENIX Secur. Symp.*, Aug. 2021, pp. 4007–4024.
- [9] L. Jiang, Q. Lou, and N. Joshi, "MATCHA: A fast and energy-efficient accelerator for fully homomorphic encryption over the torus," in *Proc. 59th ACM/IEEE Des. Automat. Conf.*, New York, NY, USA: ACM, 2022, pp. 235–240.
- [10] K. Matsuoka, S. Bian, and T. Sato, "HOGE: Homomorphic gate on an FPGA," in *Proc. 29th Asia South Pacific Des. Automat. Conf.*, Jan. 2024, pp. 325–332.
- [11] M. Van Beirendonck, J.-P. D'Anvers, F. Turan, and I. Verbauwhede, "FPT: A fixed-point accelerator for torus fully homomorphic encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA: Association for Computing Machinery, 2023, pp. 741–755.
- [12] R. Banu and T. Vladimirova, "Fault-tolerant encryption for space applications," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 45, no. 1, pp. 266–279, Jan. 2009.
- [13] S. E. Michalak, K. W. Harris, N. W. Hengartner, B. E. Takala, and S. A. Wender, "Predicting the number of fatal soft errors in los alamos national laboratory's ASC Q supercomputer," *IEEE Trans. Device Mater. Reliab.*, vol. 5, no. 3, pp. 329–335, Sep. 2005.
- [14] D. A. G. de Oliveira, L. L. Pilla, T. Santini, and P. Rech, "Evaluation and mitigation of radiation-induced soft errors in graphics processing units," *IEEE Trans. Comput.*, vol. 65, no. 3, pp. 791–804, Mar. 2016.
- [15] L. K. L. Ng and S. S. M. Chow, "SoK: Cryptographic neural-network computation," in *Proc. IEEE Symp. Secur. Privacy*, May 2023, pp. 497–514.
- [16] S. Bian et al., "HE3DB: An efficient and elastic encrypted database via arithmetic-and-logic fully homomorphic encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA: Association for Computing Machinery, Nov. 2023, pp. 2930–2944.
- [17] M. Bojarski et al., "End to end learning for self-driving cars," Apr. 2016, *arXiv:1604.07316*.
- [18] F. F. D. Santos et al., "Analyzing and increasing the reliability of convolutional neural networks on GPUs," *IEEE Trans. Reliab.*, vol. 68, no. 2, pp. 663–677, Jun. 2019.

- [19] S. Ghaznavi and C. Gebotys, "A SEU-resistant, FPGA-based implementation of the substitution transformation in AES for security on satellites," in *Proc. 10th Int. Workshop Signal Process. Space Commun.*, Oct. 2008, pp. 1–5.
- [20] M. Ayoub and W. Adi, "Fault detection and correction in processing AES encryption algorithm," in *Proc. 6th Int. Conf. Emerg. Secur. Technol.*, Sep. 2015, pp. 7–12.
- [21] S. Sau, R. Paul, and K. Maity, "Correction of MEU errors in AES using multi bit errors correction technique," in *Proc. 2nd Int. Conf. Data Sci. Bus. Analytics*, Sep. 2018, pp. 167–173.
- [22] C. N. Zhang, "Integrated approach for fault tolerance and digital signature in RSA," *IEEE Proc. - Comput. Digit. Tech.*, vol. 146, no. 3, 24 May 1999, Art. no. 151.
- [23] C. N. Zhang and C. Lai, "A systematic approach for encryption and authentication with fault tolerance," *Comput. Netw.*, vol. 45, no. 2, pp. 143–154, Jun. 2004.
- [24] L. Breveglieri, P. Maistri, and I. Koren, "A note on error detection in an RSA architecture by means of residue codes," in *Proc. 12th IEEE Int. On-Line Testing Symp.*, 2006, pp. 176–177.
- [25] J. Gava et al., "Assessment of radiation-induced soft errors on lightweight cryptography algorithms running on a resource-constrained device," *IEEE Trans. Nucl. Sci.*, vol. 70, no. 8, pp. 1805–1813, Aug. 2023.
- [26] National Institute of Standards and Technology, "Advanced encryption standard (AES)," Federal Information Processing Standards Publication 197 (FIPS PUB 197), Gaithersburg, MD, USA Nov. 26, 2001.
- [27] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978.
- [28] C. Giraud, "An RSA implementation resistant to fault attacks and to simple power analysis," *IEEE Trans. Comput.*, vol. 55, no. 9, pp. 1116–1120, Sep. 2006.
- [29] I.-C. Lin and H.-L. Wang, "An improved digital signature scheme with fault tolerance in RSA," in *Proc. 6th Int. Conf. Intell. Inf. Hiding Multimedia Signal Process.*, Oct. 2010, pp. 9–12.
- [30] C. Yakupoglu and K. Rohloff, "Towards efficient FHE based cPIR schemes and their parameter selection," in *Proc. 17th Int. Conf. Availability Rel. Secur.*, New York, NY, USA: ACM, Aug. 2022, Art. no. 136.
- [31] K. Klucznik and L. Schild, "FDFB: Full domain functional bootstrapping towards practical fully homomorphic encryption," Sep. 2021, *arXiv:2109.02731*.
- [32] L. Li et al., "ANT-ACE: An FHE compiler framework for automating neural network inference," in *>Proc. 23rd ACM/IEEE Int. Symp. Code Gener. Optim.*, New York, NY, USA: ACM, Mar. 2025, pp. 193–208.
- [33] T. Gollamudi, A. Gollamudi, and J. Gancher, "ILA: Correctness via type checking for fully homomorphic encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA: ACM, Nov. 2025, pp. 1454–1468.
- [34] J. Zhang, X. Cheng, L. Yang, J. Hu, X. Liu, and K. Chen, "SoK: Fully homomorphic encryption accelerators," *ACM Comput. Surv.*, vol. 56, no. 12, pp. 1–32, Dec. 2024.
- [35] J. Kim et al., "ARK: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse," in *Proc. 55th IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2022, pp. 1237–1254.
- [36] F. Li, M. Kumar, R. Zhu, M. Zheng, Q. Lou, and X. Xin, "ReliaFHE: Resilient design for fully homomorphic encryption accelerators," in *Proc. 31st ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, New York, NY, USA: ACM, Mar. 2026, pp. 1605–1621.
- [37] L. L. Mankali, M. Nabeel, F. Raees, M. Maniatakos, O. Sinanoglu, and J. Knechtel, "GlitchFHE: Attacking fully homomorphic encryption using fault injection," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 8481–8500.
- [38] I. Rodríguez-Ferrandez, L. Kosmidis, M. Tali, D. Steenari, A. Hands, and C. Bélanger-Champagne, "Proton evaluation of single event effects in the NVIDIA GPU Orin SoM: Understanding radiation vulnerabilities beyond the SoC," in *Proc. IEEE 30th Int. Symp. On-Line Testing Robust System Des.*, Jul. 2024, pp. 1–7.
- [39] D. C. Landauer and T. M. Lovelly, "Performance evaluation of the radiation-tolerant NVIDIA tegra K1 system-on-chip," in *Proc. IEEE Space Comput. Conf.*, Jul. 2023, pp. 24–33.
- [40] A. Serrano-Cases, S. Alcaide, M. A. Romero, Y. Morilla, and S. Cuenca-Asensi, "Analysis of kernel redundancy for soft error mitigation on embedded GPUs," *IEEE Trans. Nucl. Sci.*, vol. 70, no. 8, pp. 1700–1707, Aug. 2023.
- [41] J. M. Badia et al., "Reliability evaluation of LU decomposition on GPU-accelerated system-on-chip under proton irradiation," *IEEE Trans. Nucl. Sci.*, vol. 69, no. 7, pp. 1467–1474, Jul. 2022.
- [42] D. A. G. D. Oliveira et al., "Radiation-induced error criticality in modern HPC parallel accelerators," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, Feb. 2017, pp. 577–588.
- [43] J. E. R. Condia, B. Du, M. Sonza Reorda, and L. Sterpone, "FlexGripPlus: An improved GPGPU model to support reliability analysis," *Microelectron. Reliab.*, vol. 109, Jun. 2020, Art. no. 113660.
- [44] P. R. Bodmann, G. Papadimitriou, R. L. R. Junior, D. Gizopoulos, and P. Rech, "Soft error effects on arm microprocessors: Early estimations versus chip measurements," *IEEE Trans. Comput.*, vol. 71, no. 10, pp. 2358–2369, Oct. 2022.
- [45] T. Tsai, S. K. S. Hari, M. Sullivan, O. Villa, and S. W. Keckler, "NVBitFI: Dynamic fault injection for GPUs," in *Proc. 51st Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Jun. 2021, pp. 284–291.
- [46] S. K. S. Hari, T. Tsai, M. Stephenson, S. W. Keckler, and J. Emer, "SASSIFI: An architecture-level fault injection tool for GPU application resilience evaluation," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, Apr. 2017, pp. 249–258.
- [47] B. Fang, K. Pattabiraman, M. Ripeanu, and S. Gurumurthi, "GPU-Qin: A methodology for evaluating the error resilience of GPGPU applications," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, Mar. 2014, pp. 221–230.
- [48] F. F. d. Santos, S. K. S. Hari, P. M. Basso, L. Carro, and P. Rech, "Demystifying GPU reliability: Comparing and combining beam experiments, fault simulation, and profiling," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, May 2021, pp. 289–298.
- [49] L. L. Pilla et al., "Software-based hardening strategies for neutron sensitive FFT algorithms on GPUs," *IEEE Trans. Nucl. Sci.*, vol. 61, no. 4, pp. 1874–1880, Aug. 2014.
- [50] P. Rech, C. Aguiar, C. Frost, and L. Carro, "An efficient and experimentally tuned software-based hardening strategy for matrix multiplication on GPUs," *IEEE Trans. Nucl. Sci.*, vol. 60, no. 4, pp. 2797–2804, Aug. 2013.
- [51] JEDEC Solid State Technology Association, "Measurement and reporting of alpha particle and terrestrial cosmic ray-induced soft errors in semiconductor devices," JEDEC Standard JESD89B, Arlington, VA, USA, Sep. 2021.
- [52] S. S. Mukherjee, J. Emer, and S. K. Reinhardt, "The soft error problem: An architectural perspective," in *Proc. 11th Int. Symp. High-Perform. Comput. Archit.*, 2005, pp. 243–247.
- [53] S. S. Mukherjee, C. T. Weaver, J. Emer, S. K. Reinhardt, and T. Austin, "Measuring architectural vulnerability factors," *IEEE Micro*, vol. 23, no. 6, pp. 70–75, Nov./Dec. 2003.
- [54] R. C. Baumann, "Radiation-induced soft errors in advanced semiconductor technologies," *IEEE Trans. Device Mater. Reliab.*, vol. 5, no. 3, pp. 305–316, Sep. 2005.
- [55] F. Wang and V. D. Agrawal, "Soft error considerations for computer web servers," in *Proc. 42nd Southeastern Symp. System Theory*, Mar. 2010, pp. 269–274.
- [56] Z. Ozturk, H. R. Topcuoglu, and M. T. Kandemir, "Parallelism-based techniques for slowing down soft error propagation," in *Proc. IEEE Int. Conf. Dependable Autonomic Secure Comput., Int. Conf. Pervasive Intell. Comput., Int. Conf. Cloud Big Data Comput., Int. Conf. Cyber Sci. Technol. Congr.*, Sep. 2022, pp. 1–6.
- [57] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *J. ACM*, vol. 56, no. 6, pp. 34:1–34:40, Sep. 2009.
- [58] K. Matsuoka, Y. Hoshizuki, T. Sato, and S. Bian, "Towards better standard cell library: Optimizing compound logic gates for TFHE," in *Proc. 9th Workshop Encrypted Comput. Appl. Homomorphic Cryptography*, New York, NY, USA: Association for Computing Machinery, 2021, pp. 63–68, doi: [10.1145/3474366.3486927](https://doi.org/10.1145/3474366.3486927).
- [59] J. Mono, K. Klucznik, and T. Güneysu, "Improved circuit synthesis with multi-value bootstrapping for FHEW-like schemes," 2023. [Online]. Available: <https://eprint.iacr.org/2023/1223>
- [60] TOP500 project, Nov. Accessed: Jan. 12, 2025, 2024. [Online]. Available: <https://www.top500.org/lists/top500/2024/11/>
- [61] NVIDIA Corporation, "CUDA C programming guide," 2024, Accessed: Jan. 12, 2025. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [62] NVIDIA, "NVIDIA CUDA toolkit - free tools and training," Accessed: Jan. 18, 2023. [Online]. Available: <https://developer.nvidia.com/cuda-toolkit>

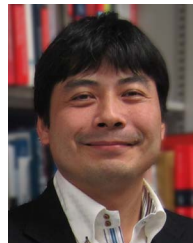
- [63] S. Hatori et al., "Developments and applications of accelerator system at the Wakasa Wan energy research center," *Nuclear Instruments and Methods in Phys. Res. Sect. B: Beam Interact. Mater. At.*, vol. 241, no. 1–4, pp. 862–869, Dec. 2005.
- [64] C. Lim, H. S. Jeong, G. Bak, S. Baeg, S.-J. Wen, and R. Wong, "Stuck bits study in DDR3 SDRAMs using 45-MeV proton beam," *IEEE Trans. Nucl. Sci.*, vol. 62, no. 2, pp. 520–526, Apr. 2015.
- [65] A. Rodriguez et al., "Proton-induced single-event degradation in SDRAMs," *IEEE Trans. Nucl. Sci.*, vol. 63, no. 4, pp. 2115–2121, Aug. 2016.
- [66] M. Yoshida, R. Iwamoto, M. Itoh, and M. Hashimoto, "Stuck errors in bits and blocks in GDDR6 under high-energy neutron irradiation," in *Proc. 2023 RADECS Data Workshop*, Sep. 2023, pp. 73–76.
- [67] NVIDIA Corporation, "NVIDIA RTX A4000 graphics card," Accessed: Apr. 5, 2023. [Online]. Available: <https://www.nvidia.com/en-us/design-visualization/rtx-a4000/>
- [68] K. Matsuoka and N. Matsumoto, "Forked cuFHE," 2020. [Online]. Available: <https://github.com/virtualsecureplatform/cuFHE>
- [69] W. Dai and B. Sunar, "cuHE: A homomorphic encryption accelerator library," in *Cryptography and Information Security in the Balkans*. Cham, Switzerland: Springer, 2016, pp. 169–186.
- [70] R. Banno and N. Matsumoto, "Iyokan: Generic engine to evaluate logical circuits on homomorphic encryption," 2020, Accessed: Jan. 26, 2023. [Online]. Available: <https://github.com/virtualsecureplatform/Iyokan>
- [71] NVIDIA, "NVIDIA nsight compute," Accessed: Dec. 12, 2024. [Online]. Available: <https://developer.nvidia.com/nsight-compute>
- [72] NVIDIA, "NVIDIA ampere GA102 GPU architecture whitepaper," 2020, Accessed: Jan. 12, 2025. [Online]. Available: <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.1.pdf>
- [73] Epoch AI, "AI training cluster sizes increased by more than 20x since 2016," 2024, Accessed: Mar. 4 2026. [Online]. Available: <https://epoch.ai/data-insights/training-cluster-size>
- [74] A. P. Putra and J.-Y. Kim, "Morphling: A throughput-maximized tfhe-based accelerator using transform-domain reuse," in *Proc. IEEE Int. Symp. High- Perform. Comput. Archit.*, 2024, pp. 249–262.
- [75] K. Matsuoka, S. Bian, and T. Sato, "HOG: Homomorphic gate on an FPGA*," in *Proc. 29th Asia South Pacific Des. Automat. Conf.*, 2024, pp. 325–332.
- [76] J. Mo, J. Gopinath, and B. Reagen, "HAAC: A hardware-software co-design to accelerate garbled circuits," in *Proc. 50th Annu. Int. Symp. Comput. Archit.*, New York, NY, USA: ACM, Jun. 2023, pp. 1–13.
- [77] Q. Lou, Y. Shen, H. Jin, and L. Jiang, "SafeNet: A secure, accurate and fast neural network inference," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [78] B. Reagen, J. Mo, and J. Gopinath, "Hardware-software co-design to accelerate garbled circuits," New York Univ., U. S. Patent no. 18/745,706, 2024.
- [79] A. T. Sheikh, A. H. El-Maleh, M. E. S. Elrabaa, and S. M. Sait, "A fault tolerance technique for combinational circuits based on selective-transistor redundancy," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 25, no. 1, pp. 224–237, Jan. 2017.
- [80] S.-S. Yau and Y.-C. Liu, "Error correction in redundant residue number systems," *IEEE Trans. Comput.*, vol. C-22, no. 1, pp. 5–11, 1973.
- [81] S. Narisada, H. Okada, K. Fukushima, S. Kiyomoto, and T. Nishide, "GPU acceleration of high-precision homomorphic computation utilizing redundant representation," in *Proc. 11th Workshop Encrypted Comput. Appl. Homomorphic Cryptography*, New York, NY, USA: Association for Computing Machinery, 2023, pp. 1–9, doi: [10.1145/3605759.3625256](https://doi.org/10.1145/3605759.3625256).



Masakazu Yoshida received the BE degree from the Faculty of Engineering, Kyoto University, Kyoto, Japan, in 2023, and the master's degree in informatics with the Kyoto University, Kyoto, Japan, in 2025. His research interests include radiation effects on GPU hardware and software.



Kotaro Matsuoka received the BE degree in electrical and electronic engineering, the master's degree in informatics, and the PhD degree in informatics from Kyoto University, Kyoto, Japan, in 2021, 2023, and 2026, respectively. He is currently an assistant professor with the Graduate School of Informatics, Kyoto University. His research interests include privacy-preserving computation, particularly fully homomorphic encryption, and hardware design for cryptographic applications.



Masanori Hashimoto (Senior Member, IEEE) received the BE, ME, and PhD degrees in communications and computer engineering from Kyoto University, Kyoto, Japan, in 1997, 1999, and 2001, respectively. He is currently a professor with the Graduate School of Informatics, Kyoto University. His research interests include design for reliability, timing and power integrity analysis, reconfigurable computing, soft error characterization, and low-power circuit design. He was the TPC chair for Asia and South Pacific Design Automation Conference (ASP-DAC) 2022 and International Midwest Symposium on Circuits and Systems (MWSCAS) 2022. He served as the editor-in-chief for *Microelectronics Reliability* (Elsevier).